

Practice notes for sorting Algorithms

- Bubblesort
 - Quicksort
 - Mergesort
-

Bubble Sort

A simple sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order.

Not very effective, especially on large data sets.

Mostly used for educational purposes as it is quite easy to trace and understand.

It's also good for showing comparisons to other sorting algorithms, which also makes it very clear to the learner why we do not use it in real code.

(source: see-algorithms.com)

QuickSort

The "Speedster" of sorting algorithms. Works by picking a "pivot" element and then arranges the rest of the elements into two groups: less than pivot to the left, greater than pivot to the right. It recursively sorts these groups, and is efficient on the largest datasets as well.

"It is a perfect blend of strategy and speed".

Performance can degrade in certain cases.

The example with 7 elements was not able to sort the list correctly when I played it. erronous visualization

Hoare Partition Scheme:

- Two pointers starting at opposite ends of array, moving towards each other
- Left pointer moves right until `element >= pivot` (does not belong on left side)
- Right pointer moves left until `element <= pivot` (does not belong on right side)
- Swaps elements
- Stops when pointers cross
- Generally does fewer swaps than Lomuto, faster in practice, but recursion logic around it needs to match its actual guarantees, which is less intuitive.

Lomuto Partition Scheme:

- Simpler and tells you exactly where the pivot ended up
- Pivot choice: always the last element in the range (`arr[high]`)
- One pointer (commonly `i` or `boundary`)
- Tracks the last index known to hold a `value < pivot`
- Starts just before the range (`low - 1`)
- Second index (`j` or `current`) scans left to right through the range checking each element against pivot

- Loops: for each element, if $< \text{pivot}$, advance boundary by one and swap into spot
- If $\geq \text{pivot}$, leave it - it will end up on the right side once pivot is placed
- Swaps the pivot into position $\text{boundary} + 1$ - the pivot's true, final sorted index
- Return value: $\text{boundary} + 1$ - the pivot's exact resting place. The pivot lives here!
- Trade-off's VS Hoare: simpler to implement and reason, tends to do more swaps (especially on arrays with duplicates)

Time Complexity (worst case): $O(n^2)$

Occurs when pivot consistently partitions the array into empty and full sub-arrays; Example: choosing the first or last element as a pivot on a list that is already 100% sorted).

Best and average Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$ (can degrade to $O(n)$ if recursion is completely unbalanced.)

(source: see-algorithms.com)

Merge Sort

Classic **Divide and Conquer** algorithm.

Splits and unsorted list into smaller sublists until each contains a single element.

The sublists are then merged together in sorted order.

Time complexity guaranteed: $O(n \log n)$.

Provides predictable performance and stability.

Pseudo code:

```
function mergeSort(start, end):  
    if start < end:  
        mid = (start + end) / 2  
        mergeSort(start, mid)  
        mergeSort(mid + 1, end)  
        merge(start, mid, end)
```

- Divides array in half
- Recursively divides each half until every sub-array contains only one element
- Merged pairs of sorted sub-arrays together
- Two sorted sub-arrays are combined by comparing their elements one at a time: smallest placed first into a temporary array
- Temporary array overwrites the corresponding section of the original array
- Continues until the entire array is reconstructed in sorted order

Useful when stability is important, and when $O(n \log n)$ worst case performance is required.

Commonly used for sorting linked lists (where its $O(n)$ space overhead does not apply), and for external

sorting when data is too large to fit in memory.

Python's built-in Timesort algorithm is a hybrid of Merge Sort and Insertion Sort, combining the best properties of both approaches.

Time complexity: Guaranteed $O(n \log n)$

Space complexity: $O(n)$

Source: <https://see-algorithms.com/sorting/MergeSort>

Insertion Sort

Simple, comparison-based sorting algorithm.

- Builds the final sorted array one element at a time
- Takes each element from the unsorted part and slices it into its correct position in the sorted part
- Effective for small data sets
- Performs significantly better than BubbleSort
- Starts with the second element in the array (considering the first element as already sorted)
- Picks this key element and compares it with the elements to its left
- If larger than key, shifts one position to the right
- Key is then inserted into the gap created by the shifting
- Only moves elements when necessary, making it efficient on already sorted data
- Ideal for small datasets, nearly sorted arrays, or as a finishing step inside more complex algorithms

Pseudo code:

```
for i = 1 to (n - 1):  
    key = arr[i]  
    j = i - 1  
    while j >= 0 and arr[j] > key:  
        arr[j + 1] = arr[j]  
        j = j - 1  
    arr[j + 1] = key
```

Time Complexity: $O(n^2)$

Best case time complexity: $O(n)$

Stability: Stable

Source: <https://see-algorithms.com/sorting/InsertionSort>

Selection Sort

Comparison-based algorithm.

Sorts an array by repeatedly finding the minimum element of the unsorted part, and moving it to its correct position.

It minimizes the number of swaps needed compared to Bubble Sort.

Useful when the cost of moving items is high, but finding the smallest item is easy.

Pseudo code:

```
for i = 0 to (n - 1):  
    min = i  
    for j = i + 1 to (n - 1):  
        if arr[j] < arr[min]:  
            min = j  
    if min != i: swap(i, min)
```

- Divides array into two logical parts
- Sorted region at the beginning
- Unsorted at the end
- Each iteration scans the entire unsorted region for the smallest element
- Smallest element is swapped with the first element of the unsorted region
- Repeats until the full array is sorted
- Does not perform multiple swaps per pass - performs a single swap only after finding the final minimum for the pass

Best suited for small arrays, or when memory writes are very expensive compared to reads.

Time complexity: $O(n^2)$ (best and worst case)

Space complexity: $O(1)$

Source: <https://see-algorithms.com/sorting/SelectionSort>

Heap Sort

An efficient sorting algorithm that leverages a data structure called **binary heap** to organize and sort data.

Reliable performance and in-place sorting capabilities.

Strong choice for handling large datasets without requiring extra memory.



heap sort

Pseudo code:

```
function heapify(i):  
    largest = i  
    left = 2 * i + 1  
    right = 2 * i + 2  
    if left < n:  
        if arr[left] > arr[largest]:  
            largest = left  
    if right < n:  
        if arr[right] > arr[largest]:  
            largest = right  
    if largest != i:  
        swap(i, largest)  
        heapify(largest)
```

Visualizer:

```
for i = (n / 2 - 1) down to 0:  
    heapify(i)  
for i = n - 1 down to 1:  
    swap(0, i)  
    heapify(0)
```

- Two main phases
- Transforms input array into a **Max Heap** (a complete binary tree where parent node > children)
- Calls a "heapify" procedure on each non-leaf node starting from the bottom of the tree
- Swaps the root with the last element of the heap
- Reduces heap size by one
- Calls heapify on the new root to restore the heap property
- Repeats until only one element remains, producing a fully sorted array

Ideal when you need guaranteed $O(n \log n)$ worst case performance with $O(1)$ extra space. Neither **Quick Sort** ($O(n^2)$ worst case)) nor **Merge Sort** ($O(n)$ extra space) can offer this. Commonly used in systems with strict memory constraints. However, **Heap Sort** is unstable and tends to have worse cache performance than **Quick Sort** due to its non-sequential memory patterns.

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$

Source: <https://see-algorithms.com/sorting/HeapSort>

Radix Sort

Organizes numbers by sorting them digit by digit.

Starts with the least significant (rightmost) and works to the most significant (leftmost).

Numbers are placed into buckets based on each digit's value, then collected back together.

Process is repeated for each digit using a stable distribution - numbers with the same digit maintain their relative order from the previous pass - leading to a fully sorted list.

Pseudo code:

```
max = largest(arr)

exp = 1

while (max / exp) > 0:

    buckets[0..9] = empty stacks

    for i = 0 to (n - 1):

        d = (arr[i] / exp) % 10

        push arr[i] to buckets[d]

    j = n - 1

    for k = 9 to 0:

        b = buckets[k]

        while b is not empty:

            arr[j] = b.pop()

            j = j - 1

    exp = exp * 10
```

- Does not directly compare two elements
- Exploits the structure of the numbers themselves
- Starting from least significant (ones place), it distributes all the numbers into 10 buckets (0-9)
- Preserves the sequence within each bucket
- Repeated for the tens place, hundreds place, and so on
- After all positions have been processed, the array is fully sorted

Excels when sorting large collections of integers or fixed-length strings where the number of digits (d) is small relative to the number of elements (n).

It outperforms comparison-based sorts in these scenarios because it avoids the $O(n \log n)$ lower bound that applies to comparison sorts.

However it is less versatile - it requires elements that can be decomposed into digits or characters.

Source: <https://see-algorithms.com/sorting/RadixSort>

All info above is sourced from <https://see-algorithms.com/> under **Sorting**.

Cheat Sheet (Google Notebook AI)

Algorithm	Best Time	Avg Time	Worst Time	Space Compx	Stable?	Key Use Case / Note
Bubble Sort 	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Pedagogical tool only.
Insertion Sort 	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Tiny list, nearly sorted list, hybrid step
Selection Sort 	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	When memory writes are extremely expensive
Merge Sort 	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Linked lists, predictable stability, external sorting
QuickSort 	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	General purpose sorting, extremely fast cache performance
Heap Sort 	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed speed with strict $O(1)$ memory limits
Radix Sort 	$O(d * (n+k))$	$O(d * (n+k))$	$O(d * (n+k))$	$O(n+k)$	Yes	Large lists of integers/fixed-length keys with small digits